Lab 4 Dead Reckoning Navigation with IMU and Magnetometer

Assigned: March 16, 2026. Due: Saturday, April 4 by 11:59pm on GitHub and Canvas

Lab learning objectives and tasks:

In this lab, you are going to build a navigation stack using two different sensors – GPS & IMU, understand their relative strengths + drawbacks, and get an introduction to sensor fusion.

Please get an early start on the lab. **You cannot complete this lab in a day!**

Collaboration:

- Data acquisition can be done collectively as a team. One dataset per team is sufficient.
- Plan data collection schedules amongst your teammates. You can talk to your teammates and ask questions on Piazza.

Hardware / Sensors

- GNSS puck - USB based, one per team. (use the one issued for Lab 1).
- VectorNav VN-100 IMU - one per team (use the one issued for Lab 3).

Software

- GPS Driver from LAB 1. It is vital that you use the driver of a team member who had written the driver correctly (correct calculations, correct header & timestamp with secs & nsecs from GPS and not system time). Verify your driver performs the correct calculations/conversions.
- IMU Driver from LAB 3. Although there were not many calculations to perform in LAB 3, ensure that this driver is also correct. For this to happen, the team should ensure the driver does the following things
  - The header frame ID is correct
  - The timestamp, both seconds & nanoseconds, **should not be** zero (MOST VITAL)
  - The VNYMR string is parsed correctly with variables correctly assigned

Part A: Collect data in a car for dead-reckoning (as a team)

Logistics

1. If any teammate has access to a car, it would be more convenient for data collection so that you can do it on your team's schedule.
2. Alternatively, a team can schedule to use our autonomous car 'NUANCE' for data collection. There will be a sign-up sheet on Piazza for you to reserve a time slot.

Setup & Data Collection

Before embarking, test everything by collecting data from both the GPS and IMU sensors. Make sure you can see and record both gps msgs and imu msgs on your machine. Decide on a roughly 10 minute driving loop that starts and ends at Ruggles station. Be ready to share this route with your driver.

1. To collect data from both the GPS and IMU sensors, create a single launch file that launches both your GPS & IMU driver nodes, and make sure you are able to see both *gps* and *vectornav* topics on your machine. The two topics should exist & be publishing data together successfully.

- You can verify data is being published by opening multiple terminal windows and doing <u>rostopic echo gps</u> and <u>rostopic echo imu</u> in separate windows. To ease things, we suggest installing terminator by *<u>sudo apt-get install terminator</u>*. It will allow you to have multiple terminal screens in one window instead of switching tabs or windows.

2. Before data collection (keep the car turned off until specified)

- Mount the IMU inside your vehicle with electrical tape on the center of the car's dashboard. If you are using the Nuance Car, then mount the IMU on the mount in the cup holder between the two front seats. Make sure that the x–axis is pointed forward and that the IMU is horizontal (verify with a leveling app on your phone). Add some electrical taps as a strain relief so the IMU does not change orientation if the cable is tugged.
- Fix the GPS puck to the roof – it has a magnetic back and should just stay there. Open the window slightly to lead the cable into the car.

- Connect both sensors to one single laptop and launch the two nodes. These two nodes should never die during the data collection (more explained in part 3), so do not turn them off. If a USB unplugs or if one of the nodes crashes, you must collect your entire dataset again. We suggest one member monitors the screen to ensure that topics are being published.

3. You will collect two datasets in this lab

- The Car Circles
  - Begin a rosbag and call it *data_going_in_circles.bag*
  - Wait 10 - 15 seconds
  - Start your car
  - Drive the car in circles 4-5 times (the more circular the path, the better). One suggested place is the Ruggles circle near Centennial common.
  - Stop recording only the rosbag. **Do not** turn off the car or the driver nodes. If NUANCE's engine stops (it's a hybrid), that's OK.

- Mini Boston Tour
  - Begin a rosbag and call it *data_driving.bag*.
  - You might want to make a video of your route as you drive, it may help correlate any data anomalies with external events.
  - Wait 10 - 15 seconds. To get data with the car stationary.
  - Go for a drive around Boston **AND** return to the spot where you started. We suggest a total distance of **AT LEAST** 2 – 3 kilometers with a **MINIMUM** of 10 turns.
  - As you drive continuously check that you are recording data. If data recording stops then you have to go back to your starting point and try again.
    - Note: if you successfully collected your Car Circles data then you don't have to repeat that part of the experiment.
  - Do not go underground/in tunnels (e.g., eastbound on Columbus to go underneath Mass Ave). Enjoy the view (whether outdoors or on the Linux screen).
  - Once you return to the spot you started from (loop closure), stop the rosbag

recording

- You can now turn off your car & your ROS drivers.

Note: Please follow all local & state traffic laws. Always yield to pedestrians. And remember that not everyone likes robots.

Part B: Analysis of the data collected in Part A (Individually)

1. **Estimate the heading (yaw)**

Magnetometer Calibration:

- Correct magnetometer readings for "hard-iron" and "soft-iron" effects using the data collected when going around in circles. **Do not** use *magcal* (a built-in function of MATLAB). Write the code yourself. The *magcal* function is not optimized for high-grade sensors.

- *Submit a plot showing the magnetometer data before and after the correction in your report. Use your driving in a circle data.*

Sensor Fusion:

All further analysis will now be performed with data from data_driving.bag and the previously obtained calibration will be used for the magnetometer.

- Calculate the yaw angle *in degrees* from the magnetometer calibration & plot the raw magnetometer yaw with the corrected yaw for comparison. Going forward, only use the corrected magnetometer yaw.

- Integrate the yaw rate from the gyro sensor to get yaw angle. One way to do this is to use '*trapz*' or '*cumtrapz*' commands in MATLAB and treat your time series as a function that is being integrated. You are free to use your own function if the results are better.

- Compare the yaw angle from above two methods. (Magnetometer vs. Yaw Integrated from Gyro).

- Use a complementary filter to combine the yaw measurements from the magnetometer and yaw rate from gyro as described in class (Dead Reckoning Lecture, slides 54-56) to get an improved estimate of the yaw angle. Filter the magnetometer estimate using a low-pass filter and filter the integrated gyro data using a high-pass filter. Look at plots of the raw data to determine reasonable values for the cutoff frequencies of your filters. You might also find the '*unwrap*' or '*wraptopi*' commands in MATLAB useful.

- Plot the results of the low pass filter (magnetometer), high pass filter (integrated gyro) and complementary filter together on one graph.

- Compare your sensor fusion yaw result with the yaw angle computed by the IMU and write down your observations. *Note: you* ***cannot*** *use the original yaw (Euler angle) from the IMU in any analysis except this last part.*

Graphs to submit:

- The magnetometer X-Y plot before and after hard and soft iron calibration
- The time series magnetometer data before and after the correction in your report.
- Low-Pass filtered magnetometer, High-Pass filtered integrated gyro data, and Complementary Filter plots together in one graph.
- Compare the yaw angle between four methods: Magnetometer, Yaw Integrated from Gyro, Complementary filter, Yaw angle computed by the IMU (Euler angle). Use single plot graphs or multiple plot graphs as you see fit.

2. Estimate the forward velocity

1. Integrate the forward acceleration to estimate the forward velocity. Your initial values for velocity may be too low or high. You may need to make some adjustments on these data (e.g., is there velocity in another axis? Do the values make sense given your knowledge of what the car was doing?)

2. Calculate an estimate of the velocity from your GPS measurements.

Plots to submit:

- Velocity estimate from the GPS

- Velocity estimate from accelerometer before and after adjustment

Dead Reckoning with IMU

- Integrate the forward velocity to obtain displacement and compare with GPS displacement.

- We simplify the description of the motion by assuming that the vehicle is moving in a two-dimensional plane.

  Denote the position of the center-of-mass (CM) of the vehicle by (X,Y,0) and its rotation rate about the CM by (0,0,ω). We denote the position of the inertial sensor in space by (x,y,0) and its position in the vehicle frame by (x_c,0,0).

  Then the acceleration measured by the inertial sensor (i.e. its acceleration as sensed in the vehicle frame) is

$$\ddot{x}_{obs} = \ddot{X} - \omega\dot{Y} - \omega^2 x_c$$
$$\ddot{y}_{obs} = \ddot{Y} + \omega\dot{X} + \dot{\omega} x_c$$

  where all of the quantities in these equations are evaluated in the vehicle frame.
  Assume that $\dot{Y} = 0$ (that is, the vehicle is not skidding sideways) and ignore the offset by setting $x_c = 0$ (meaning that the IMU is on the center of mass of the vehicle, i.e. the point about which the car rotates). Then the first equation above reduces to $\ddot{X} = \ddot{x}_{obs}$.

  Integrate this to obtain $\dot{X}$. Compute $\omega\dot{X}$ and compare it to $\ddot{y}_{obs}$. How well do they agree? If there is a difference, what is it due to? Can you fix it? Plot all results.

- With the previous assumptions, use the heading from the magnetometer (or your complementary filter if you have better results) to rotate your previously estimated forward velocity.

  Denote this vector by (v_e,v_n) , where 'e' means Easting and 'n' means Northing.

Integrate (v_e,v_n) to estimate the trajectory of the vehicle (x_e,x_n).

Compare this estimated trajectory with the GPS track by plotting them on the same graph. Make sure to adjust starting point, so that both the tracks start at the same point and same heading (adjust heading so that the first straight line from both are oriented in the same direction).
Report any scaling factor used for comparing the tracks.

Questions to answer in your report

1. How did you calibrate the magnetometer from the data you collected? What were the sources of distortion present, and how do you know?
2. How did you convert raw magnetometer readings (gauss) to angles (degrees).
3. How did you use a complementary filter to develop a combined estimate of yaw? What components of the filter were present, and what cutoff frequency(ies) did you use? Show the equation of your complementary filter.
4. Which estimate or estimates for yaw would you trust for navigation? Why?
5. What adjustments did you make to the forward velocity estimate, and why?
6. What discrepancies are present in the velocity estimate between accel and GPS. Why?
7. Compute $\omega\dot{X}$ and compare it to $\ddot{y}_{obs}$. How well do they agree? If there is a difference, what is it due to?
8. Estimate the trajectory of the vehicle (x_e,x_n) from inertial data and compare with GPS. (adjust heading so that the first straight lines from both are oriented in the same direction). Report any scaling factor used for comparing the tracks.
9. Given the specifications of the VectorNav, how long would you expect that it is able to navigate without a position fix? For what period of time did your GPS and IMU estimates of position match closely? (within 2 m) Did the stated performance for dead reckoning match actual measurements? Why or why not?

Grading

Chapter 1, Yaw estimation	**40**
Graph: The magnetometer X-Y plot before and after hard and soft iron calibration	5
Graph: The time series magnetometer data before and after the correction	5
Graph. Low-Pass filtered magnetometer, High-Pass filtered integrated gyro data, and Complementary Filter plots together in one graph.	5
Graph. Compare the yaw angle between four methods: Magnetometer, Yaw Integrated from Gyro, Complementary filter, Yaw angle computed by the IMU (Euler angle). Use single plot graphs or multiple plot graphs as you see fit.	5
Answers to questions 1 - 4	20
Chapter 2. Estimation of Forward Velocity	**20**
Graph. Plots of forward velocity from acceleration before and after adjustment	5
Graph. Plots of adjusted forward velocity and GPS	5
Answer to questions 5 and 6	10
Chapter 3. Dead Reckoning	**30**
Diagram of trajectory calculated from dead reckoning overlayed with trajectory obtained from GPS data.	15
Answers to questions 7, 8, and 9	15
Professional engineering style used in report	5
Complete graphs with correct axes labels, legend if necessary and title.	5
Total	**100**

How to Submit Lab_4

1. In your class repo 'EECE5554', create a directory called LAB4
2. Copy the ROS driver package used for this assignment under LAB4.
3. Inside LAB4, create sub-directories 'data' and 'analysis'
4. Copy the MATLAB/python code used for data analysis and dead reckoning under 'analysis'
5. Place your report in pdf format also in analysis directory.

   Your repo structure should look similar to

   '<Path_to_repo>/EECE5554 /LAB4/src/<your_driver_files>'
   '<Path_to_repo>/EECE5554 /LAB4/data/<your data files>'
   '<Path_to_repo>/EECE5554 /LAB4/src/analysis/<your analysis files>'
   '<Path_to_repo>/EECE5554 /LAB4/src/analysis/report.pdf'

6. Push your local commits to (remote) GitHub server. You can verify this by visiting gitlab.com and making sure you can see the commit there.
7. Upload your report to the Lab 4 entry in Assignments on Canvas

Repo Structure:

EECE5554				
	LAB4			
		src		
			gps_driver	
				driver files (only 1 team member)
			imu_driver	
				driver files (only 1 team member)
			Custom message folder	
				msg folder containing GPSmsg.msg and IMUmsg.msg, and other files. ((only 1 team member))
		data		
			ROS2 bag files (only 1 team member)	
		analysis		
			Analysis files	
			Report.pdf	
		README.md		

Note: Please mention the Github username of the team member in the report who is uploading their drivers to Github. Entire group will be evaluated using those drivers. Please add a readme.md for instructions on how to run your code (launch both nodes with a single launch file).